

QA AUTOMATION FRAMEWORK

Module Testing & Automation Guide

How a module is tested, from Jira story to Allure report, and what developers need to provide for reliable automated tests.

Audience	Developers, QA engineers, product owners, team leads
Applies to	Web applications, mobile web, native Android and iOS apps
Tooling	Playwright, Appium (WebdriverIO), Claude Code + Playwright MCP, Allure 3
Version	1.0 · 2 October 2026

Contents

1	Purpose and who does what	Start here
2	The testing process, step by step	QA & developers
3	The module MD file: what developers must provide	Developers
4	Test ID contract (web and mobile)	Developers
5	Definition of Ready checklist	Everyone
6	Worked example: a complete module MD	Developers
7	From test cases to automation	QA
8	Running tests and reading the Allure report	QA & developers
9	Failure triage and bug reporting	QA & developers
10	When a module changes after testing	QA & developers
11	Things that block automation, and how to avoid them	Developers
A	Appendix: blank module MD template	Copy & fill
B	Appendix: glossary and commands	Reference

The one-sentence version

For every module, the developer hands QA one Markdown file that lists every field, rule, message and test ID (Section 3 and the template in Appendix A). QA turns it into reviewed test cases, Claude turns those into automated tests, and the results appear in the Allure report linked to the Jira story.

1. Purpose and who does what

This guide describes how every module of our web and mobile applications is tested with the shared automation framework, and what information has to be agreed *before* testing starts. Automation quality depends on the quality of its input: most flaky or wrong tests come from vague requirements ("show an error") or from UI elements that can't be located reliably. A complete module MD file solves both.

1.1 Roles

Role	Responsibilities
Developer	Writes or completes the module MD file (fields, rules, exact messages, test IDs, APIs, test data). Adds <code>data-testid</code> / accessibility IDs in the code. Deploys the build to the QA environment. Fixes confirmed bugs.
Product owner / BA	Owns the Jira story and acceptance criteria. Answers open questions about business rules.
QA engineer	Checks the MD file against the Definition of Ready. Generates and reviews test cases, runs automation, triages failures, raises bugs, signs off the module.
Claude (AI assistant)	Drafts test cases from the MD file, explores the app through the Playwright MCP browser, writes the model, page, data and spec code, runs tests and proposes fixes. Its output is always reviewed by QA.

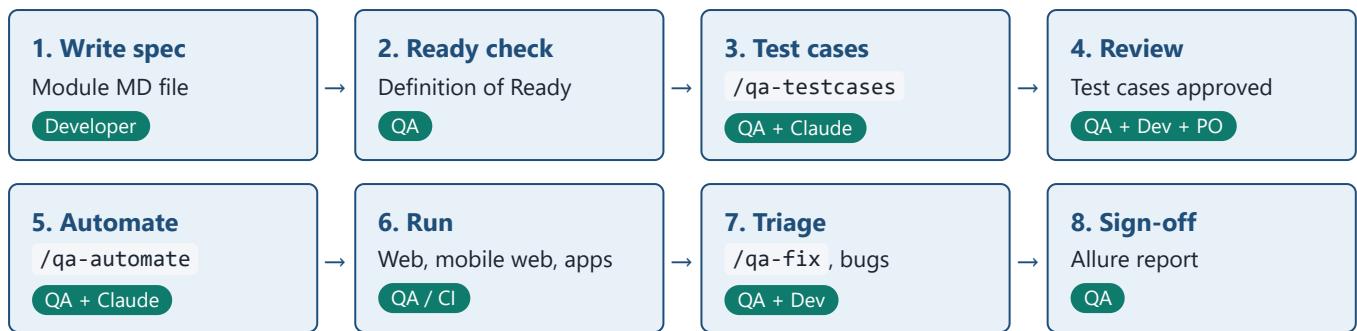
1.2 Where things live in the repository

Folder / file	Contains	Owner
<code>apps/<app>/app.config.ts</code>	The application: platforms, QA URL, test-ID attribute, login roles	QA, from developer input
<code>apps/<app>/requirements/</code>	Module MD files: <code><JIRA-KEY>-<module>.md</code>	Developer + QA
<code>apps/<app>/test-cases/</code>	Reviewed test cases: <code><module>.testcases.md</code>	QA
<code>apps/<app>/models/</code>	Field models: each field's label, type, locator and rules	QA (generated)
<code>apps/<app>/pages/</code> , <code>screens/</code>	Page / screen objects with business actions	QA (generated)
<code>apps/<app>/data/</code>	Typed test data per module	QA
<code>apps/<app>/tests/</code>	Automated test specs, one test per test-case ID	QA (generated)
<code>apps/<app>/.env</code> (never committed)	QA URLs, test accounts per role, app builds, API token	QA, from developer input
<code>src/</code>	The shared framework used by every application (no app code here)	Framework owners

1.3 One framework, many applications

Every project uses the same framework. A new application is added with `npm run new:app -- <name> --platforms web,android`, which creates `apps/<name>/`. The developer and QA then fill in `app.config.ts` (QA URL, test-ID attribute, user roles and how to log in) and the app's `.env` (test accounts, app builds). The active application is chosen with the `APP` setting. Improvements to the framework reach every project at once.

2. The testing process, step by step



Step 1: Developer writes the module MD file

As soon as the story is in development, the developer copies `templates/requirement.md` to `apps/<app>/requirements/<JIRA-KEY>-<module>.md` and fills it in (Section 3 explains every part). Free-form notes are accepted, but the information in Section 3 must be in there.

Output: module MD file committed or shared with QA. **Done when:** it passes the Definition of Ready (Section 5).

Step 2: QA checks the Definition of Ready

QA reads the file against the checklist in Section 5. Missing exact messages, missing test IDs or unclear rules go back to the developer as comments on the Jira story. **Automation does not start on an incomplete spec:** guessing produces tests that check the wrong thing.

Step 3: Generate test cases

In VS Code, QA runs `/qa-testcases apps/shop/requirements/SCRUM-123-registration.md`. Claude reads the file, optionally opens the page in a browser to confirm fields and messages, and writes `apps/shop/test-cases/registration.testcases.md` containing:

- at least one **positive** case for every acceptance criterion;
- **negative** cases for every required field and every validation rule;
- **boundary** cases for lengths and ranges (min-1, min, max, max+1);
- cases for **business rules**, **roles/permissions** and **UI states** (empty, disabled, error);
- IDs `TC-<MODULE>-NN`, tags `@smoke` (main happy path) or `@regression`, and whether each case can be automated.

Claude also lists **gaps and questions**, for example an error message that is not specified.

Step 4: Review the test cases

QA reviews the test-cases file and shares it with the developer and PO (in the PR or the Jira story). This is the cheapest place to catch misunderstandings: a wrong expected result fixed here never becomes a wrong test.

Step 5: Automate

QA runs `/qa-automate apps/shop/test-cases/registration.testcases.md`. Claude:

1. opens the application through the **Playwright MCP** browser and confirms locators and message texts (web);
2. creates the **field model**, **page/screen object**, **test data** and **spec** (Section 7);
3. type-checks and runs the new tests, fixing automation problems until they pass;
4. reports any test that fails because the application behaves differently from the spec (a possible bug).

QA reviews the generated code like any pull request.

Step 6: Run on every platform

Web specs run on desktop Chrome and on mobile emulation (Pixel 7, iPhone 15). Native app specs run on Android and iOS through Appium. The same tests run in CI on every deployment to the QA environment.

Step 7: Triage failures

QA runs `/qa-fix` for failing tests. Each failure is classified as an **automation issue** (fixed in the framework), an **application bug** (Jira bug raised, test marked as expected-to-fail with the bug key), or an **unclear requirement** (question to PO). See Section 9.

Step 8: Sign-off

The module is signed off when every automatable test case passes (or is linked to an accepted, open bug) on all target platforms. The Allure report link is attached to the Jira story.

Typical timeline for one module

Spec ready with the code → test cases same day → review within 1 day → automation and first run within 1–2 days → regression on every later build at no extra cost.

3. The module MD file: what developers must provide

One file per module (or per story that changes a module). Use the template in `templates/requirement.md` (reproduced in Appendix A). Sections marked **required** must be complete before QA starts. The table below explains what each section is used for, so it's clear why the detail matters.

Section	What to write	How automation uses it
1. Overview required	Jira key, epic, module, platform, QA URL/path, build or app version, feature flag, priority, names.	Allure grouping (Epic → Feature → Story), Jira link in the report, which test projects run, the base URL.
2. User story required	As a ... I want ... so that ...	Story label in the report; context for test-case generation.
3. Acceptance criteria required	Numbered (AC1, AC2...), testable, Given/When/Then, exact expected texts in quotes.	Each AC becomes at least one test case; traceability AC → test ID.
4. Entry point & preconditions required	URL or menu path, roles that can access it, data that must exist first.	Page path, which login session to use, setup steps.
5. Fields required	Every input, dropdown, checkbox, button and label with type, test ID, required, default, rules, dependencies and exact error message.	Becomes the field model directly; drives negative and boundary cases; error texts become assertions.
6. Actions & outcomes required	Each button/link: when it is enabled, what happens on success (navigate, toast, close dialog) and on failure.	Page methods (<code>submit()</code> , <code>cancel()</code>) and their expected results.
7. Messages required	Every toast, banner, dialog and empty state, with its test ID and exact text including punctuation.	The <code>Messages</code> constant in the model; text assertions.
8. Business rules	Calculations, limits, role permissions, conditional fields, uniqueness (e.g. "email must be unique").	Dedicated rule test cases; data-driven tests for calculations.
9. UI states	Loading, empty, server-error, disabled/read-only behaviour.	State tests, often using API mocking.
10. APIs	Endpoints the screen calls, with method, purpose, success and error codes.	Fast test setup and cleanup via API; mocking error states; checking that the right request is sent.
11. Test data & accounts required	Accounts per role (password as an <code>.env</code> variable name, never the value), seed data, cleanup method, OTP/captcha/email bypass.	Roles in <code>app.config.ts</code> , accounts in the app <code>.env</code> , data files; API cleanup keeps tests repeatable.
12. Mobile only	APK/IPA link, package/bundle ID, launch activity, OS versions, permissions, deep links.	Appium capabilities in <code>.env</code> ; deep links skip slow navigation.
13. Out of scope / known issues	What is not delivered yet; known bugs with Jira keys.	Prevents false failures; known bugs become expected-to-fail tests.
14. Change log	Date, what changed, who.	Tells QA which test cases and models to update.

3.1 The fields table in detail

This is the most important part of the file. Each row becomes one entry in the field model.

Column	Meaning	Good example
Field	Visible label, as the user sees it	Email address
Type	One of: text, password, email, number, textarea, date, dropdown, checkbox, radio, file, button, link, label	email
Test ID	The <code>data-testid</code> (web) or accessibility ID (mobile); see Section 4	<code>register-email-input</code>
Required	yes / no / conditional (explain in "Depends on")	yes
Default	Pre-filled or pre-selected value	India (dropdown)
Rules	Min/max length, format (regex or description), numeric range, allowed options, file types and size	max 100 chars, valid email format, unique
Depends on	Shown, enabled or required only when another field has a value	Required when Country = "United States"
Error message	Exact text per rule; give each rule its own message	required: "Email is required"; format: "Enter a valid email address"

Not testable

- "Email should be validated."
- "Show an error if the password is weak."
- "Phone number field with proper format."
- "Button should work."

Testable

- Email: required, max 100, RFC format, unique. Errors: "Email is required", "Enter a valid email address", "This email is already registered".
- Password: 8–64 chars, at least 1 upper, 1 lower, 1 digit. Error: "Password must be 8–64 characters and include upper case, lower case and a number".

4. Test ID contract (web and mobile)

Automated tests find elements by a dedicated, stable attribute rather than CSS classes, XPath or visible text, which change with styling, refactoring and translations. Adding test IDs is a small development task that makes automation **much** more stable. Every element listed in the fields, actions and messages tables needs one.

4.1 Naming convention

<module>-<element>-<kind> kebab-case, lower case, English, no spaces

register-email-input	register-submit-button	register-email-error
register-country-select	register-terms-checkbox	register-success-toast
orders-row-<orderId>	orders-row-<orderId>-delete-button	(lists: append the record's stable id)

Kind suffix	Use for
-input	text, email, password, number, date, textarea
-select	dropdowns / pickers
-checkbox , -radio , -switch	toggles (for radio groups put the ID on the group, and give each option a unique ID or label)
-button , -link	actions and navigation
-error	the inline validation message of a field
-toast , -banner , - dialog	feedback messages and modals
-title , -label , - value	read-only text that tests check
-row , -card , -list	repeated items and their container

4.2 How to add them, per technology

Technology	Code
HTML / React / Angular / Vue	<code><input data-testid="register-email-input" /></code>
React Native	<code><TextInput testID="register-email-input" accessibilityLabel="register-email-input" /></code>
Flutter	Semantics(identifier: 'register-email-input', child: ...) or Key('register-email-input') with the Appium Flutter driver
Android (XML / Compose)	android:id="@+id/register_email_input" and/or android:contentDescription ; Compose: Modifier.testTag("register-email-input") with testTagsAsResourceId = true
iOS (UIKit / SwiftUI)	view.accessibilityIdentifier = "register-email-input" ; SwiftUI: .accessibilityIdentifier("register-email-input")

4.3 Rules

- **Stable:** never generated (no `input-4821` , no hashed class names) and not changed during refactoring.
- **Unique** on the screen. In lists, append the record's business ID, not its position.
- **Not translated:** test IDs stay English when the UI language changes.
- **Present in QA/staging builds.** They may be stripped from production if required, but never from test builds.
- On mobile, use the **same IDs on Android and iOS** when possible, so one model serves both platforms.
- The attribute name (`data-testid` , `data-test` , `data-qa`) is configurable per application (`testIdAttribute` in the app's `app.config.ts`); pick one per application and use it everywhere.

No test IDs yet?

The framework can fall back to accessible roles and labels (for example "the button named *Register*"), which Claude finds through the Playwright MCP. This works, but breaks when texts change or the app is translated. Treat missing test IDs as technical debt to be fixed in the next sprint.

5. Definition of Ready checklist

QA checks a module MD file against this list before generating test cases. Every unchecked item goes back to the developer or PO as a Jira comment.

Scope and access

- ☐ Jira key, epic, module and platform(s) filled in
- ☐ QA environment URL / app build available and working
- ☐ Feature flag named, with how to switch it on
- ☐ Entry point (URL, menu path or deep link) given
- ☐ Roles that can and cannot access the module listed

Behaviour

- ☐ Acceptance criteria numbered and testable
- ☐ Every button has success *and* failure outcomes
- ☐ Business rules have concrete numbers and limits
- ☐ Conditional fields and their trigger values described
- ☐ Loading, empty and error states described

Fields and messages

- ☐ Every field has type, required flag and rules
- ☐ Lengths, ranges and formats are specific numbers or patterns
- ☐ Dropdown / radio options listed in display order
- ☐ Every validation rule has its own **exact** error text
- ☐ Every toast, banner and dialog text is exact

Testability

- ☐ Every element in the tables has a test ID (Section 4)
- ☐ Test accounts exist for every role (passwords via `.env`)
- ☐ Way to create and clean up test data (API or script)
- ☐ OTP, captcha and email have a test bypass
- ☐ Mobile: APK/IPA, package/bundle ID and devices given

6. Worked example: a complete module MD

Below is what a "ready" file looks like for a customer registration screen. It is long because it is complete: every line here removes a guess from the tests.

How to read this example

The file is written in Markdown (headings with `##`, tables with `|` pipes, as in `templates/requirement.md`). It is shown formatted here so it's easier to read.

SCRUM-123 — Customer registration

1. Overview

Jira	SCRUM-123	Epic	Customer onboarding
Module	Registration	Platform	web, mobile-web
Environment / URL	https://qa.shop.example.com/register	Build	web 4.12.0
Feature flag	none	Priority	critical
Developer / QA	A. Developer / B. Tester		

2. User story

As a new visitor, I want to create an account so that I can place orders.

3. Acceptance criteria

1. **AC1** Given all fields are valid, when I click "Create account", then my account is created, I see the toast "Welcome, {firstName}!" and I land on `/account`.
2. **AC2** Given an email that is already registered, then I see "This email is already registered" under the email field and no account is created.
3. **AC3** "Create account" stays disabled until "I accept the terms" is checked.
4. **AC4** Users younger than 18 cannot register.

4. Entry point & preconditions

- URL `/register`, or the "Sign up" link on the login page (test ID `login-signup-link`).
- Logged-out users only; logged-in users are redirected to `/account`.
- Country list is loaded from `GET /api/v1/countries`.

5. Fields

#	Field	Type	Test ID	Req	Default	Rules	Depends on	Error message (exact)
1	First name	text	register-firstname-input	yes		2–50 chars; letters, space, ' and -		required: "First name is required" format: "Use letters only"
2	Last name	text	register-lastname-input	yes		1–50 chars; same as first name		required: "Last name is required"
3	Email	email	register-email-input	yes		max 100; email format; unique		required: "Email is required" format: "Enter a valid email address" unique: "This email is already registered"

#	Field	Type	Test ID	Req	Default	Rules	Depends on	Error message (exact)
4	Mobile number	text	register-mobile-input	no		exactly 10 digits		format: "Enter a 10-digit mobile number"
5	Country	dropdown	register-country-select	yes	India	from /api/v1/countries, sorted A–Z		
6	State	dropdown	register-state-select	cond.		US states	shown and required if Country = United States	required: "State is required"
7	Date of birth	date	register-dob-input	yes		DD/MM/YYYY; age ≥ 18		required: "Date of birth is required" age: "You must be at least 18 years old"
8	Password	password	register-password-input	yes		8–64 chars; ≥1 upper, ≥1 lower, ≥1 digit		"Password must be 8–64 characters and include upper case, lower case and a number"
9	Confirm password	password	register-confirm-input	yes		equals Password		"Passwords do not match"
10	I accept the terms	checkbox	register-terms-checkbox	yes	unchecked			
11	Create account	button	register-submit-button			disabled until terms checked		
12	Field errors	label	register-<field>-error			one per field, under the field		

6. Actions & outcomes

Action	Test ID	Enabled when	On success	On failure
Create account	register-submit-button	terms checked	toast "Welcome, {firstName}!", go to /account	inline field errors, stay on page
Already have an account? Log in	register-login-link	always	go to /login	

7. Messages

Key	Where	Test ID	Exact text
welcome	toast	register-success-toast	Welcome, {firstName}!
serverError	banner	register-error-banner	Something went wrong. Please try again.

8. Business rules

- **BR1** Email comparison is case-insensitive (Test@x.com is the same as test@x.com).
- **BR2** Leading and trailing spaces are trimmed from all text fields before validation.

9. UI states

- Loading: the submit button shows a spinner (`register-submit-spinner`) and is disabled.
- Server error (5xx): `serverError` banner; entered values are kept.

10. APIs

Method	Endpoint	Purpose	Success	Errors
POST	<code>/api/v1/customers</code>	create account	201	409 duplicate email, 422 validation
DELETE	<code>/api/v1/test/customers/{email}</code>	cleanup (QA env only)	204	

11. Test data & accounts

Purpose	Username / data	Secret (.env variable)	Notes
Existing customer	<code>existing.customer@example.com</code>	<code>EXISTING_USER_PASS</code>	for AC2 duplicate check
New registrations	<code>qa.<random>@example.com</code>	—	deleted with the cleanup API

No OTP or captcha in the QA environment.

13. Out of scope / known issues

- Social sign-up (Google) is a separate story, SCRUM-130.

14. Change log

Date	Change	By
2026-10-01	First version	A. Developer

In Markdown, the first fields row is written as:

```
| 1 | First name | text | register-firstname-input | yes | | 2-50 chars; letters, space, ' and - | |  
required: "First name is required"; format: "Use letters only" |
```

6.1 What QA generates from it (excerpt)

ID	Title	Type	Tags	From
TC-REG-01	Valid registration shows welcome toast and opens /account	positive	@smoke	AC1
TC-REG-02	Already-registered email shows "This email is already registered"	negative	@regression	AC2
TC-REG-03	Email check is case-insensitive	rule	@regression	BR1
TC-REG-04	Create account disabled until terms accepted	ui	@regression	AC3
TC-REG-05	Age 17 years 364 days is rejected; exactly 18 is accepted	boundary	@regression	AC4
TC-REG-06	Each required field left empty shows its own message	negative	@regression	Fields
TC-REG-07	Password 7 / 8 / 64 / 65 characters	boundary	@regression	Fields
TC-REG-08	State required only when Country = United States	rule	@regression	Fields
TC-REG-09	Server error keeps entered values and shows banner	ui	@regression	UI states

7. From test cases to automation

Every part of the MD file has exactly one place in the code. This is why reviewers can check generated code against the spec line by line.

MD file section	Code artefact	Example
Overview (Jira, epic, module)	<code>storyInfo({...})</code> in the spec	Allure: Customer onboarding → Registration → SCRUM-123
Entry point	<code>page path</code>	<code>readonly path = '/register'</code>
Fields table	<code>field model apps/shop/models/web/registration.model.ts</code>	one entry per row: label, type, locator, rules
Messages table	<code>RegistrationMessages</code> constant in the model	<code>welcome: (name) => `Welcome, \${name}!`</code>
Actions & outcomes	<code>page methods</code>	<code>register(data), expectWelcome(name)</code>
Test data & accounts	<code>apps/shop/data/web/registration.data.ts</code> + <code>app .env</code>	valid customer, invalid cases array
Test cases file	<code>spec apps/shop/tests/web/registration/registration.spec.ts</code>	one test per ID, titled <code>TC-REG-01 ...</code>
APIs	<code>api + cleanup fixtures, act.mockApi</code>	create data via API and delete it after the test; fake a 500 for the error state
Roles	<code>auth.roles</code> in <code>app.config.ts</code>	<code>test.use({ role: 'admin' })</code>

7.1 The four layers

```
// 1. MODEL: the fields table, as data
export const RegistrationFields = defineWebFields({
  email: {
    label: 'Email', type: 'email', locator: { testId: 'register-email-input' },
    rules: { required: true, maxLength: 100,
      messages: { required: 'Email is required', pattern: 'Enter a valid email address' } },
  },
  terms: { label: 'I accept the terms', type: 'checkbox', locator: { testId: 'register-terms-checkbox' } },
  submit: { label: 'Create account', type: 'button', locator: { testId: 'register-submit-button' } },
});
// ... one entry per row
export type RegistrationData = FormData<typeof RegistrationFields>; // typed automatically

// 2. PAGE: business actions, no locators
export class RegistrationPage extends BasePage {
```

```

readonly path = '/register';
readonly fields = RegistrationFields;
async register(data: RegistrationData) {
  await this.act.fillForm(this.fields, data);    // fills each field according to its type
  await this.act.click(this.fields.submit);
}
}

// 3. DATA: typed by the model
export const validCustomer: RegistrationData = { firstName: 'Asha', email: uniqueEmail(), terms: true,
/* ... */ };

// 4. SPEC: reads like the test case
test('TC-REG-01 | Valid registration shows welcome toast and opens /account',
  { tag: ['@smoke', '@TC-REG-01'] }, async ({ registrationPage }) => {
    await registrationPage.register(validCustomer);
    await registrationPage.expectWelcome(validCustomer.firstName!);
  });

```

7.2 What appears in the report

Every action is a readable step, so a failed test can be understood without opening the code:

```

TC-REG-01 | Valid registration shows welcome toast and opens /account
  Open "/register"
  Fill "First name" with "Asha"
  Fill "Email" with "qa.lx3k9a@example.com"
  Fill "Password" with "*****"          <- passwords are always masked
  Check "I accept the terms"
  Click "Create account"
  Verify "Welcome toast" has text "Welcome, Asha!"

```

7.3 Mobile modules

Native screens follow the same pattern: `defineMobileFields`, a screen object extending `BaseScreen`, and `this.act.tap / type / fillForm / swipe`. If Android and iOS use different IDs, a field can declare both: `locator: { android: { id: '...' }, ios: { accessibilityId: '...' } }`. The Playwright MCP cannot drive native apps, so mobile locators come from the MD file (Section 12 and the test ID column) or from Appium Inspector.

8. Running tests and reading the Allure report

8.1 Commands

Command	What it runs
<code>npm test</code>	Everything. Native app tests are skipped if no app is configured.
<code>npm run test:web</code>	Web specs on desktop Chrome
<code>npm run test:mobile-web</code>	Web specs on Pixel 7 and iPhone 15 emulation
<code>npm run test:android / test:ios</code>	Native app specs through Appium
<code>npm run test:smoke / test:regression</code>	By tag
<code>npx playwright test --grep @TC-REG-02</code>	One test case by ID
<code>npm run test:ui / test:headed / test:debug</code>	Interactive runner / visible browser / step-through debugging
<code>TEST_ENV=staging npm test</code>	Against another environment (<code>apps/<app>/env.staging</code>)
<code>npm run report</code>	Build and open the Allure report

8.2 Reading the report

Results tree

Results grouped by **Epic** → **Feature** → **Story**, matching Jira. Use it for module sign-off. Role logins appear under *Test setup*.

Test detail

The description shows the **test case** (preconditions, steps, expected result) from the test-cases file, then the readable steps, Jira link, severity and tags.

Categories tab

Failures sorted by likely cause: **Product bugs** (developers start here), **Automation issues** (QA) and **Environment problems**.

History, dashboard

Run history shows flaky tests and regressions. The report's start page also links a Dashboard with charts and trends.

What a failed test shows

Every failed test opens with a summary (likely cause, the step that failed, expected vs actual) and has a **Bug report** attachment, generated automatically for every application:

- **Expected vs Actual:** what the test checked, the test case's expected result and what the app showed;
- **Steps to reproduce:** every executed step with the real data (passwords masked) and the failing step marked;
- **Details:** Jira story and test case, severity, environment and URL, browser/version or device, user role, page URL at failure, attempt;
- **Browser console errors** and **failed network requests** (4xx/5xx) captured during the test;
- the error and Playwright call log, and a **Copy for Jira** text block;
- next to it: screenshot, video and trace (web), or screenshot and page source (mobile).

The likely cause is a first guess

It is based on the failing step and error (wrong text, wrong page, element not found, server unreachable). When the test checked a different text than the test case specifies, the failure is marked as an *automation issue*, not a product bug. QA confirms the cause before a Jira bug is raised (Section 9).

8.3 Severity

The story's priority in the MD file becomes the Allure severity: **blocker** (release stopper), **critical** (core flow broken), **normal**, **minor**, **trivial**. A release needs zero failing blocker and critical tests.

9. Failure triage and bug reporting

Each failing test is classified before anything is changed. `/qa-fix` helps: Claude re-runs the test, reproduces it in the browser and proposes a classification, and QA confirms it.

Classification	Typical signs	Action
Automation issue	Element not found after a UI change, timing, expired test data, environment down	Fix the model locator, data or setup. Never weaken an assertion to make a test pass.
Application bug	Behaviour differs from the MD file / test case: wrong message, wrong navigation, data not saved	Raise a Jira bug (copy the <i>Copy for Jira</i> text from the Bug report). Add <code>knownBug('SCRUM-456', 'summary')</code> as the first line of the test: it stays green while the bug exists, links the bug in the report, and turns red once the bug is fixed, so QA removes the line.
Spec changed	Developer changed behaviour on purpose, but the MD file wasn't updated	Developer updates the MD file and change log; QA updates test cases and the model.
Unclear requirement	Both behaviours seem reasonable	Ask the PO; the test stays as it is until answered.

9.1 Bug report contents

- **Title:** [Registration] Duplicate email accepted (TC-REG-02)
- **Story / test case:** SCRUM-123, TC-REG-02
- **Environment:** QA, web 4.12.0, Chrome desktop (and others where it reproduces)
- **Steps:** from the Bug report's *Steps to reproduce* (or its *Copy for Jira* block)
- **Expected:** from the MD file, with the exact text. **Actual:** what happened.
- **Evidence:** Allure report link, screenshot, trace or video

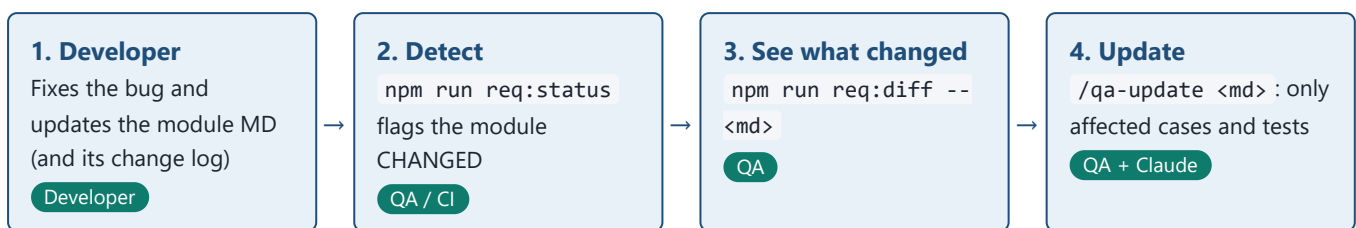
10. When a module changes after testing

Modules change after QA has tested them: a bug fix adds a field, a validation message is reworded, a new acceptance criterion appears. The framework records which version of each module MD file QA last processed (the **baseline**), so changes are found automatically. Nobody has to compare files by eye or remember what was tested.

10.1 Module stages

Stage	Meaning	Recorded by
Test cases written	Test cases generated and reviewed for this version of the MD	<code>/qa-testcases</code> (automatic)
Automated	All automatable test cases pass for this version	<code>/qa-automate</code> , <code>/qa-update</code> (automatic)
Signed off	QA has verified the module at this version	QA only: <code>npm run req:baseline -- <md> --stage signed-off --by "Name"</code>

10.2 The re-test loop



Example: status

Module MD	Jira	Automated	Last QA stage	Attention
web/SCRUM-101-login.md	SCRUM-101	5/5	Signed off 2026-10-02	CHANGED since baseline
web/SCRUM-102-products.md	SCRUM-102	5/5	Signed off 2026-10-02	

Example: what changed

Acceptance criteria
+ ADDED AC5 After 3 wrong passwords the account is locked for 15 minutes with "Too many attempts..."
Fields
~ CHANGED Password: Rules: "masked" → "masked, 8-64 chars";
Error message: "Password is required" → "Password must be at least 8 characters"
+ ADDED Remember me (Type: checkbox; Required: no; Test ID: remember-me)

10.3 What `/qa-update` does with the changes

Change in the MD	Effect on test cases and automation
Field / AC / rule / message added	New test cases with the next free IDs; model, page, data and specs extended
Rule or exact text changed	Affected test cases updated, same IDs ; assertions updated

Change in the MD	Effect on test cases and automation
Behaviour removed	Test cases retired (Automate: no , kept for history); their tests removed
Priority changed	Severity and tags updated

It also re-runs the module's smoke tests (a fix can break the happy path), adds an entry to the test-cases file's **Change history**, records the new baseline as *Automated*, and gives QA a **re-test list**. QA re-tests and signs off.

For developers

- Always edit the **same** module MD file. Don't create a new copy per iteration, or the history is lost.
- Rows are matched by their first column (field name, action, message key). A renamed field therefore shows as removed + added; write the rename in the change log ("Mobile" renamed to "Phone number") so QA keeps the same test cases.
- Add a change-log line (section 14) with the Jira bug key, e.g. 2026-10-09 | Password min length 8 (SCRUM-456) | A. Dev .

Baselines are stored in `apps/<app>/requirements/.baseline/` and are committed with the repository, so the whole team (and CI) sees the same status.

11. Things that block automation, and how to avoid them

Problem	Effect on tests	Developer-side solution
No test IDs, or generated IDs	Brittle locators that break on every UI change	Add test IDs per Section 4
Vague or missing messages	Tests can only check "some error", so real bugs slip through	Exact text per rule in the MD file
OTP / SMS / captcha	Cannot be automated reliably	Fixed OTP or bypass in QA env, or a test API that returns the OTP; captcha disabled for test accounts
Email verification links	Tests stuck waiting for an inbox	Test inbox API (e.g. Mailpit/Mailhog) or an endpoint that returns or verifies the link
Shared or changing test data	Tests pass alone and fail together	Dedicated accounts per role; create and delete data via API
No cleanup endpoint	Database fills up; unique-value rules start failing	QA-only cleanup API or nightly reset script
Animations, long loaders	Slow, flaky tests	Loading indicators with test IDs; reduced-motion respected
Third-party widgets (payments, maps) in iframes	Slow, unstable, may hit real services	Sandbox mode and documented test cards; feature flag to stub
Behaviour depends on date/time or location	Tests fail on certain days or machines	Document the rule; allow time and locale override in QA env
Mobile: no debuggable build	Appium cannot inspect the app	Provide QA builds (debug/ad-hoc signed) with accessibility IDs
Spec not updated after a change	Tests fail on correct behaviour	Update the MD file and change log in the same PR as the code

Golden rule for developers

If a person needs to ask you a question to know whether the screen behaves correctly, the answer belongs in the module MD file.

Appendix A: Blank module MD template

Also in the repository as `templates/requirement.md`. Copy it to `apps/<app>/requirements/<JIRA-KEY>-<module>.md`.

```
# <JIRA-KEY> - <Module / story title>

## 1. Overview (required)
| Item | Value |
| ----- | ----- |
| Jira | <JIRA-KEY> |
| Epic | <Epic / product area> |
| Module | <Module name> |
| Platform | web / mobile-web / android / ios |
| Environment / URL | <QA URL + path> |
| Build / version | <web build or app version; APK/IPA link> |
| Feature flag | <flag + how to enable, or "none"> |
| Priority | blocker / critical / normal / minor |
| Developer / QA | <names> |

## 2. User story (required)
As a <role>, I want <goal> so that <benefit>.

## 3. Acceptance criteria (required)
1. AC1 Given ..., when ..., then ...

## 4. Entry point & preconditions (required)
- How to reach the screen:
- Roles that can access it:
- Data that must exist first:

## 5. Fields (required)
| # | Field | Type | Test ID | Required | Default | Rules | Depends on | Error message (exact) |
| - | - | - | - | - | - | - | - | - |

## 6. Actions & outcomes (required)
| Action | Test ID | Enabled when | On success | On failure |
| ----- | ----- | ----- | ----- | ----- |

## 7. Messages (required)
| Key | Where shown | Test ID | Exact text |
| --- | --- | --- | --- |

## 8. Business rules
- BR1:

## 9. UI states
- Loading: - Empty: - Error (5xx): - Disabled / read-only:

## 10. APIs (setup, cleanup, mocking)
| Method | Endpoint | Purpose | Success | Errors |
| ----- | ----- | ----- | ----- | ----- |

## 11. Test data & accounts (required)
| Purpose | Username / data | Secret (.env variable) | Notes |
| ----- | ----- | ----- | ----- |
- Cleanup:
- OTP / captcha / email:

## 12. Mobile only
| Item | Android | iOS |
```

App build			
Package / bundle id			
Launch activity		n/a	
Min OS / test devices			
Permissions asked			
Deep link to this screen			

13. Out of scope / known issues

-

14. Change log

Date	Change	By
----	-----	--

Appendix B: Glossary and commands

Term	Meaning
Module MD file	The Markdown specification of one module or story, written by the developer (Section 3).
Test-cases file	Reviewed list of test cases derived from the module MD file; one row = one automated test.
Field model	Code description of a page's fields (label, type, locator, rules); the only place locators live.
Page / screen object	Class with business actions for one web page or mobile screen (<code>register()</code> , <code>expectError()</code>).
Test ID	Stable attribute used to find an element: <code>data-testid</code> on web, accessibility ID on mobile.
Playwright	Browser automation framework used for web and mobile-web tests, and as the test runner for everything.
Appium	Automation server for native Android/iOS apps, driven through WebdriverIO.
MCP / Playwright MCP	Lets Claude control a real browser to explore pages, find locators and confirm behaviour.
Allure	Test report grouped by Epic/Feature/Story, with steps, attachments, Jira links and history.
Smoke / regression	Smoke: a few critical happy paths run on every build. Regression: the full set.
Definition of Ready	Checklist a module MD file must pass before testing starts (Section 5).

Claude commands (VS Code)

Command	Result
<code>/qa-testcases <requirements md></code>	Structured test cases in <code>apps/<app>/test-cases/</code> , plus open questions
<code>/qa-automate <test-cases md></code>	Model, page/screen, data and spec, run until green
<code>/qa-fix [spec @tag]</code>	Failure triage: automation fixes and suspected bugs

One-time setup

```
npm install
npm run setup          # browsers
copy .env.example .env # root: APP, TEST_ENV, JIRA_BASE_URL
npm run new:app -- shop --platforms web,android # new application, then fill
                                     # apps/shop/app.config.ts and apps/shop/.env
claude mcp add playwright --scope user -- cmd /c npx -y @playwright/mcp@latest ^
  --isolated --output-dir=.playwright-mcp
```